A Comparative Report on the Loop Scheduling Options in OpenMP

Exam No. B024703

October 27, 2016

1 Introduction

This report details experiments undertaken to compare the performance of the loop scheduling options available in OpenMP. A serial code involving two separate loops was provided to the Author. This code measured and reported the time taken for each loop to execute 1000 times.

The first loop (loop 1) iterates through the elements of a two dimensional array, calculating the cosine of each element and assigning this value to another similarly sized array. The second loop (loop 2) performs more operations per element in a similarly sized array to those in loop 1. This time a sum of varying length is performed for every element in the two dimensional array, with each term in the sum requiring the calculation of a logarithm.

The nature of these loops led to different performance characteristics under parallelisation for each. This is due to the varying complexities of calculation within and between each of these loops. The variation in amounts of operations performed per loop cycle can cause issues with load balancing, where some threads complete all assigned work before others, leading to idling and sub-optimal performance.

The purpose of the loop scheduling options provided by OpenMP is to provide a range of strategies to improve load balancing between different threads. The remainder of this report will describe the methods used to measure the effectiveness of each of the loop schedules before detailing the results of these

experiments. Finally, this report will present a conclusion drawn from these results.

2 Methods

2.1 Compiling and Running Serial Code

As a precaution, to check the correctness of any subsequent parallel version, the original code was compiled in its serial form. This was done using the Intel C Compiler with the `-O3` option specified at compile time for maximum serial optimisation. Beyond using this to verify the correctness of parallel solution and that parallel performance was credible, analysis of the performance of this serial code falls outside the scope of this report.

2.2 Parallelising the Code

Before different scheduling clauses could be used, the code had first to be parallelised. This was done using OpenMP directives and apart from these directives involved no alterations to the original source code. Within the directive for launching a parallel for loop, OpenMP gives the option of setting a schedule. The available schedule options are as follows:

- `static`
- `auto`
- `static,n`
- `dynamic,n`
- `guided,n`

Where `n` specifies a chunksize. The Intel documentation [1] describes how the programmer has the option to set the schedule using an environment variable. This was done so that the schedule could be changed programmatically and that all experiments would be easily repeatable.

2.3 Compiling and Running the Code

As with the serial case the code was compiled using the Intel C Compiler and using the `-O3` optimisation flag. In this case, however, the appropriate compiler flag was used to compile an OpenMP program. To measure the performance of the different scheduling options the code was initially run on 4 threads on a backend node of `cirrus`, an Intel Xeon powered cluster. For the schedules where it is possible to set a chunksize, this chunksize took the values 1, 2, 4, 8, 32 and 64.

From these results, the scheduling option with the best performance for each loop on 4 threads was determined. Using these two options, the best for loop 1 and the best for loop 2, the code was then run again on 1, 2, 4, 6, 8, 12 and 16 threads. This allowed the measurement of the speed up, as defined by T_p/T_1, so that the scalability of the best performing schedule on 4 threads could be analysed.

For all measurements the running time was measured multiple times and a mean of these values used. In addition the standard deviation was also calculated. This allows the comparison of performance variability between different schedules. Performance variability is often an important consideration in HPC applications, where it may be desirable to avoid methods that often, but unreliably, yield high performance.

3 Results

3.1 Correctness

For almost all HPC applications correctness is the primary concern. The serial code provided included a check sum that could be used to verify the correctness of future parallel results. In all parallel cases this sum yielded the same result as the serial code. We can therefore disregard correctness as a factor when comparing the different schedules.

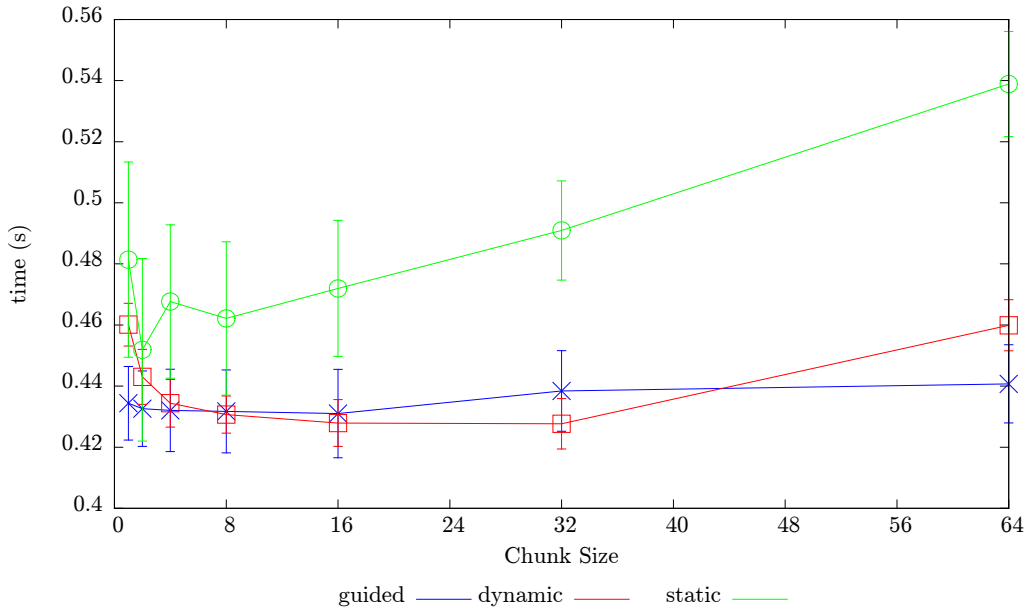

Figure 1: Graph showing execution time for loop 1 against block size for the guided, dynamic and static schedules. The error bars show 1 standard deviation. Note that these error bars are much larger than in loop 2.

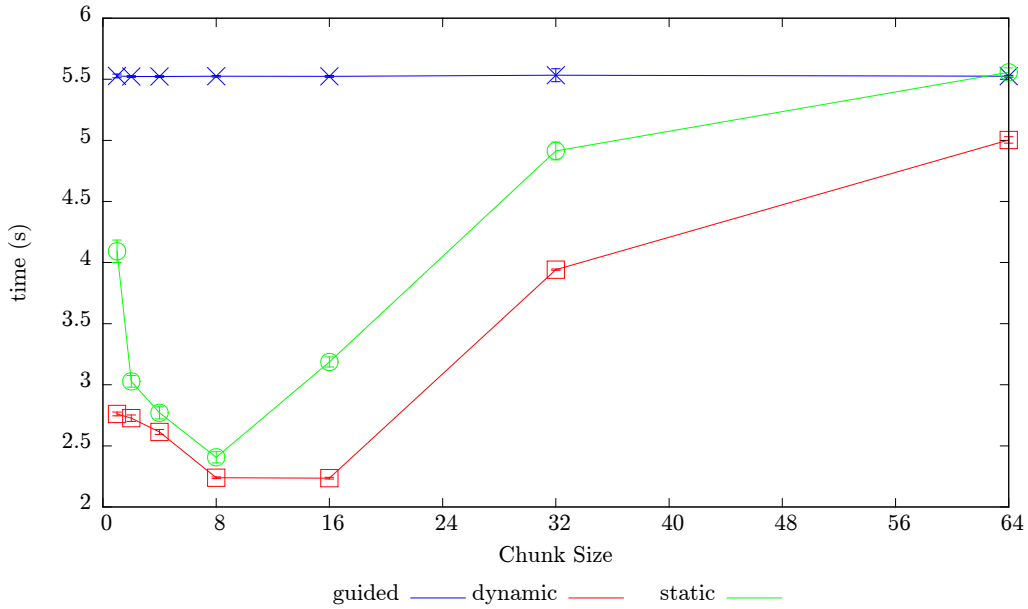

Figure 2: Graph showing execution time for loop 2 against chunk size for the guided, dynamic and static schedules.

3.2 Results of Schedules on 4 Threads

The first results gathered were execution times for each loop for every available loop schedule. The results for these are shown in Figures 1 and 2.

For the loop 1 results there was a large standard deviation in all the schedules and chunk sizes. This shows a wide performance variability for this loop, particularly where the `static` schedule was used. For loop 2 much less variable results were achieved as shown by the small error bars showing standard deviation in Figure 2.

The highest performing schedule for loop 1 was found to be `dynamic,32`. This had the lowest mean execution time and also had one of the lowest standard deviations, meaning this schedule had reliably good performance for this loop.

The highest performing schedule for loop 2 was not immediately obvious. The mean execution time with the `dynamic,8` and `dynamic,16` was similar to the order of nanoseconds - well beyond the accuracies of this experiment. It was therefore decided that both of these schedules would be jointly selected

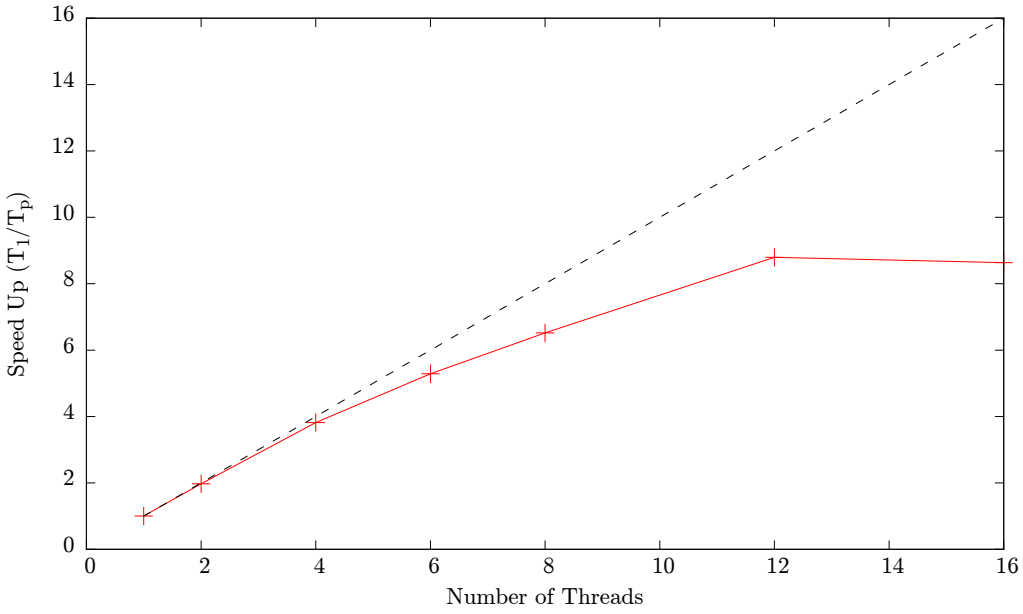

Figure 3: Speed Up of loop 1 for the dynamic 32 schedule. A theoretical ideal speed up is shown as a black dotted line.

as the best.

3.3 Speed Up of Best Schedules

After the best schedules for loop 1 and loop 2 had been found they were then run again on 1, 2, 4, 6, 8, 12 and 16 threads. The speed up for loop 1 on the `dynamic,32` schedule is shown in Figure 3. Here relatively good scalability is shown until the system reaches 12 cores. At this point no more speed up is achieved.

4 conclusion

The best way to do it was:

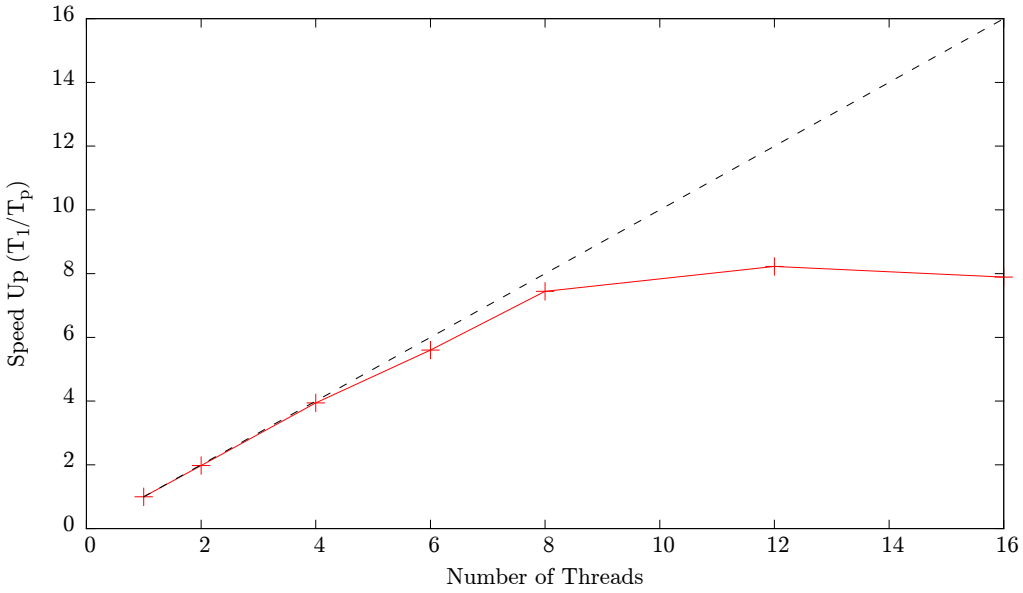

Figure 4: Speed Up for the dynamic 8 schedule. A theoretical ideal speed up is shown as a black dotted line.

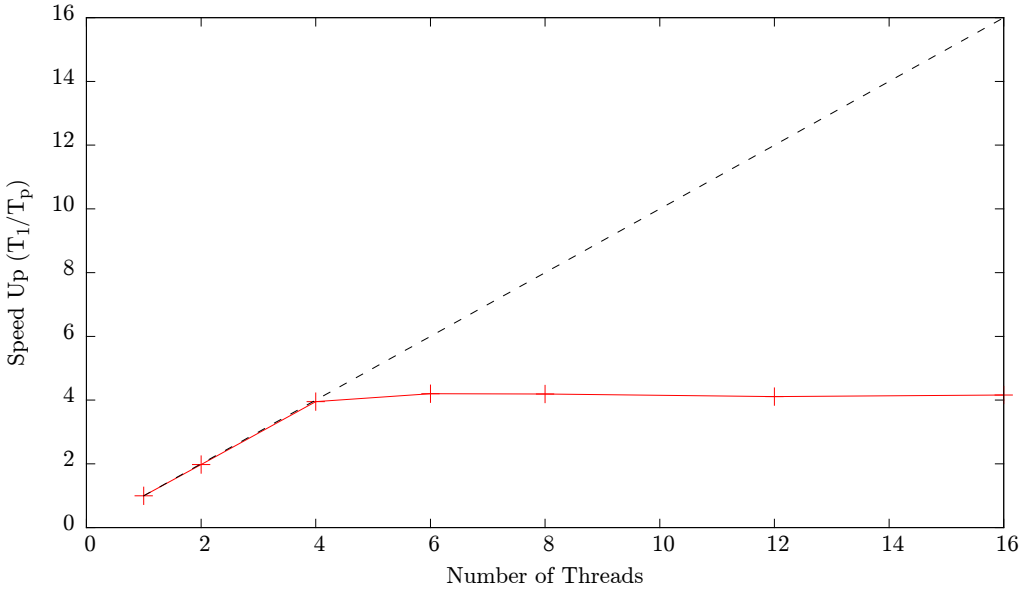

Figure 5: Speed Up for the dynamic 16 schedule. A theoretical ideal speed up is shown as a black dotted line.

References

[1] Intel Corp. *OpenMP Loop Scheduling* `https://software.intel.com/en-us/articles/openmp-loop-scheduling` Accessed: 26-10-2016

A Full Results

Table 1: Loop 1

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
static	4	0.728231	0.00666976
auto	4	0.430124	0.00860091
static,1	4	0.481388	0.0319373
static,2	4	0.451891	0.0298839
static,4	4	0.467681	0.0250977
static,8	4	0.462157	0.0250929
static,16	4	0.471986	0.0222526
static,32	4	0.490942	0.0162345
static,64	4	0.538883	0.0171833
dynamic,1	4	0.460126	0.0069964
dynamic,2	4	0.443026	0.00900873
dynamic,4	4	0.434388	0.00780827
dynamic,8	4	0.430674	0.00608488
dynamic,16	4	0.42792	0.00766377
dynamic,32	4	0.427667	0.0082635
dynamic,64	4	0.459919	0.00837936
guided,1	4	0.434381	0.0120444
guided,2	4	0.432622	0.0123446
guided,4	4	0.432052	0.0134753
guided,8	4	0.431721	0.0135889
guided,16	4	0.43101	0.0144585
guided,32	4	0.438409	0.0131864
guided,64	4	0.440735	0.0127809
dynamic,8	1	1.64584	0.00368912
dynamic,8	2	0.842835	0.000531856
dynamic,8	4	0.432655	0.0101219
dynamic,8	6	0.300801	0.0106516
dynamic,8	8	0.229031	0.00784543
dynamic,8	12	0.159318	0.00782003
dynamic,8	16	0.126953	0.00697052
dynamic,16	1	1.6453	0.000578072
dynamic,16	2	0.837223	0.00103113
dynamic,16	4	0.432996	0.0128874
dynamic,16	6	0.303278	0.0114546
dynamic,16	8	0.23483	0.00944744
dynamic,16	12	0.162625	0.00981424
dynamic,16	16	0.138232	0.00830332

Table 2: Loop 2

Schedule	No. Threads	Mean Time (s)	Std. Deviation (s)
static	4	6.35165	0.0350881
auto	4	5.5247	0.0176731
static,1	4	4.09267	0.0922924
static,2	4	3.02777	0.0478233
static,4	4	2.7703	0.0496694
static,8	4	2.4068	0.0457846
static,16	4	3.18633	0.0415496
static,32	4	4.91394	0.0732637
static,64	4	5.55721	0.0367112
dynamic,1	4	2.76134	0.0153146
dynamic,2	4	2.72649	0.0267236
dynamic,4	4	2.61394	0.0205074
dynamic,8	4	2.23896	0.007317
dynamic,16	4	2.23559	0.00661029
dynamic,32	4	3.94226	0.00563607
dynamic,64	4	5.00349	0.0262898
guided,1	4	5.52794	0.0151542
guided,2	4	5.52264	0.00682783
guided,4	4	5.52289	0.00714884
guided,8	4	5.52478	0.00667864
guided,16	4	5.52349	0.00793705
guided,32	4	5.5337	0.0532817
guided,64	4	5.52464	0.00727738
dynamic,32	1	8.84485	0.0103624
dynamic,32	2	4.46239	0.00300086
dynamic,32	4	3.94598	0.00589791
dynamic,32	6	3.94177	0.00235488
dynamic,32	8	3.9493	0.00520081
dynamic,32	12	3.98098	0.0194448
dynamic,32	16	3.95387	0.00761888